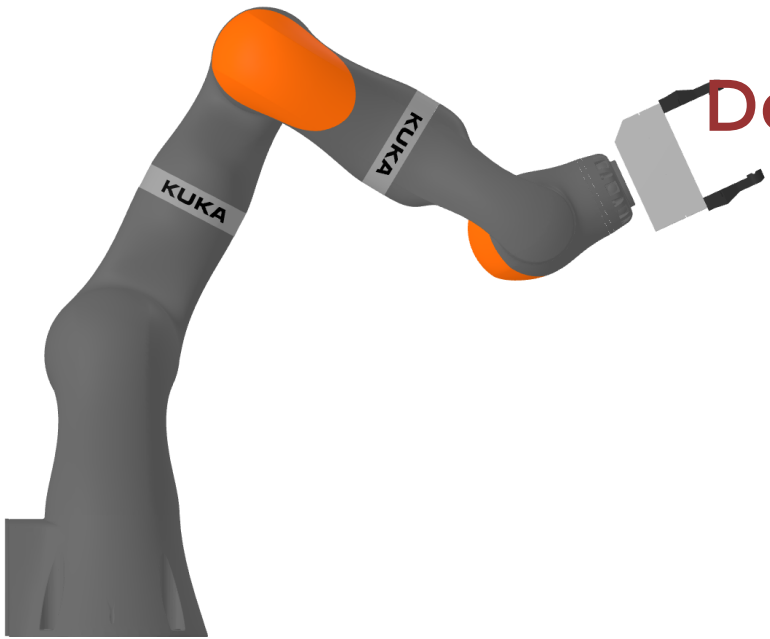


Design and Simulation in Drake

MIT 6.4210/2: Robotic Manipulation

Fall 2025, Lecture 2

Massachusetts Institute of Technology

Reminders

- Read the course web page https://manipulation.csail.mit.edu/Fall2025/index.html
- Sign up for Piazza
- Sign up for Gradescope
  - PS 1 due Thursday 10pm
- 6.4210 students
  - Sign up for CI-M recitation on Canvas
- 6.4212 students
  - Read about extra pset (in Schedules page)

What is Drake?



CORE LIBRARY

Modeling Dynamical Systems	Solving Mathematical Programs	Multibody Kinematics and Dynamics
API | TUTORIAL	API | TUTORIAL	API

drake.mit.edu

Drake features

- Systems framework
- Deterministic simulations
- Optimization framework
- Sophisticated contact modeling (hydroelastic)

Contact simulation

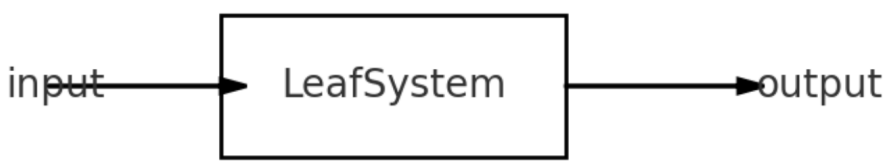




Systems Framework

Drake models computations as Systems, wired together in a Diagram.

- LeafSystem: user-defined input/output and dynamics.
- Diagram: composition of multiple subsystems.
- Context: stores state, time, parameters, inputs.
- Ports: inputs and outputs that connect systems.



Example: Simple LeafSystem

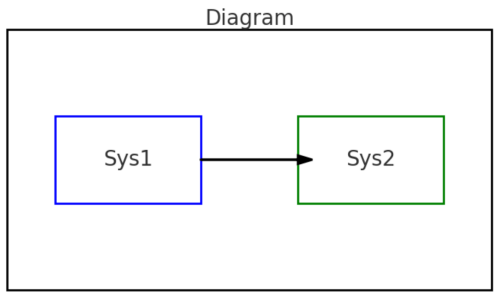
```
class DoubleSystem(LeafSystem):
    def __init__(self):
        super().__init__()
        self.DeclareVectorInputPort('u', 1)
        self.DeclareVectorOutputPort('y', 1, self.CalcY)

    def CalcY(self, context: Context, output: BasicVector) -> None:
        u = self.get_input_port('u').Eval(context)
        output.SetFromVector([2*u[0]])
```

- a = BasicVector(3) # 3-d vector
- a.SetFromVector(array) # from numpy array
- a.CopyToVector() # to numpy array

Diagram Composition

- Diagrams allow composition of multiple subsystems.
  - Each system has input/output ports.
  - Ports are connected to build larger systems.




```
builder = DiagramBuilder()
builder.AddSystem(sys1)
builder.AddSystem(sys2)
builder.Connect(sys1.get_output_port(0), sys2.get_input_port(0))
diagram = builder.Build()
```

Dynamic systems

```python
class VerticalMassUnderGravity(LeafSystem):
    """
    Continuous-time 1D vertical mass:
        ż = v
        v̇ = g + f/m   (constant gravity)
    State vector: x = [z, v]
    """
    def __init__(self, gravity=-9.81, mass=1.0):
        super().__init__()
        self._g = float(gravity)
        self._m = float(mass)

        # Continuous state: q (z), v (ż)
        self.DeclareContinuousState(2)
        # Input force f
        self.DeclareVectorInputPort("force", 1)

        # Output: position
        self.DeclareVectorOutputPort("position", 1, self.CalcPosition)

    def CalcPosition(self, context: Context, output: BasicVector) -> None:
        x = context.get_continuous_state_vector()
        output.SetFromVector([x[0]])

    def DoCalcTimeDerivatives(self, context:Context, derivatives:ContinuousState) -> None:
        x = context.get_continuous_state_vector().CopyToVector()
```

Simulation

```
if __name__ == "__main__":

    builder = DiagramBuilder()
    mass_sys = builder.AddSystem(VerticalMassUnderGravity())

    # Two ways of doing the logger
    logger = LogVectorOutput(mass_sys.get_output_port(0), builder)
    # logger = builder.AddSystem(VectorLogSink(1))
    # builder.Connect(mass_sys.get_output_port(0), logger.get_input_port(0))

    diagram = builder.Build()

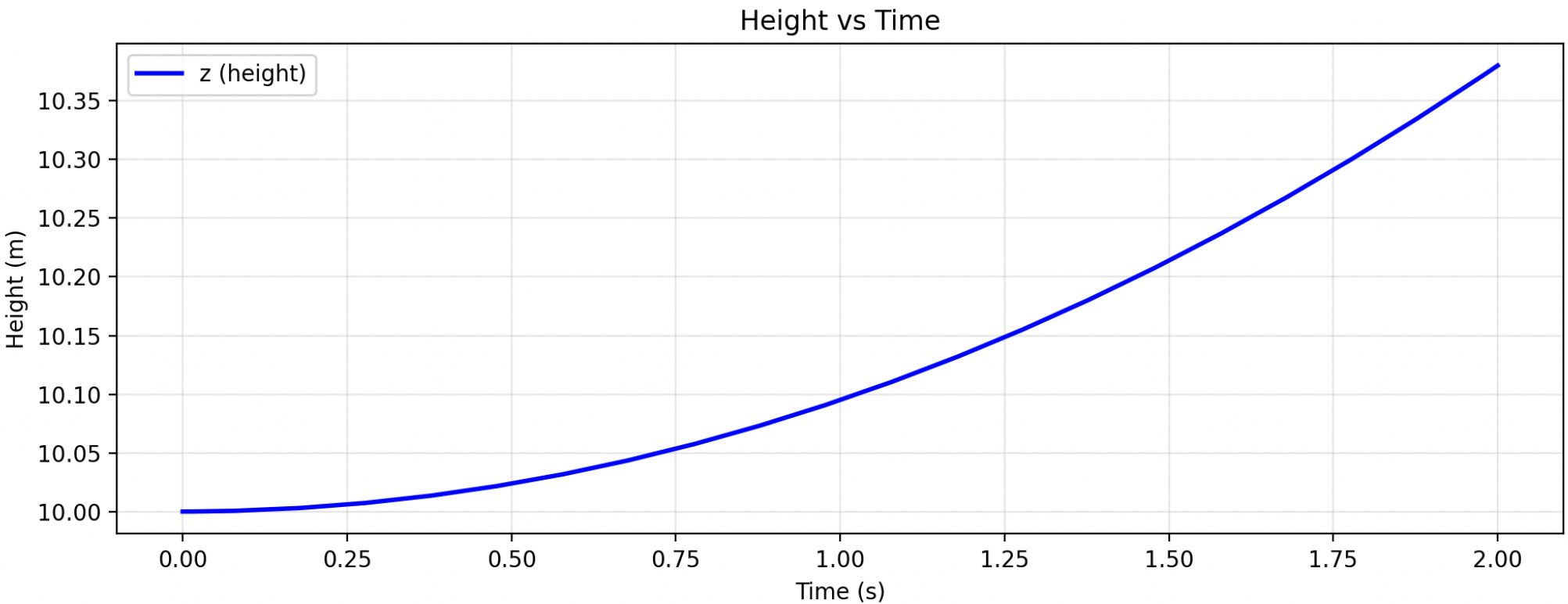
    ctx = diagram.CreateDefaultContext()
    sim = Simulator(diagram, ctx)

    ctx.get_mutable_continuous_state().SetFromVector([10.0, 0.0])
    sys_ctx = mass_sys.GetMyContextFromRoot(ctx)
    mass_sys.get_input_port(0).FixValue(sys_ctx, 10.0)

    sim.AdvanceTo(2.0)  # simulate 2 seconds
    z_final = mass_sys.get_output_port(0).Eval(sys_ctx)

    print("Final position [z] =", z_final)

    plot(sim)
```


Height vs Time
z (height)
Height (m)
10.35
10.30
10.25
10.20
10.15
10.10
10.05
10.00
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
Time (s)

Drake's Physics Engine: MultibodyPlant

MultibodyPlant is Drake's class for modeling robots and mechanisms.

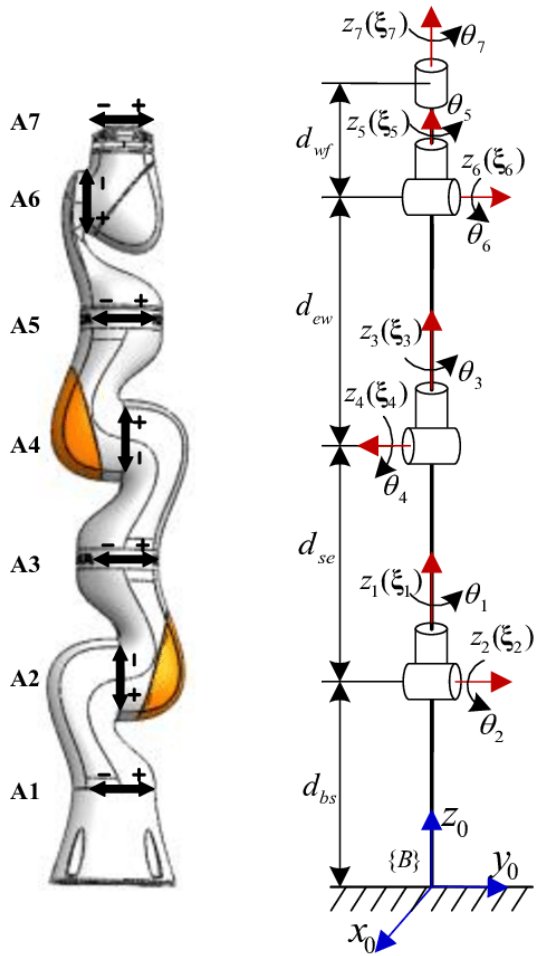
- Add bodies, joints, actuators, forces.
- Register geometry with SceneGraph for visualization/collision.
- Parse models from URDF/SDF files.
- Requires Finalize() after model is built to produce System

```
builder = DiagramBuilder()
plant, scene_graph = AddMultibodyPlantSceneGraph(builder, time_step=1e-4)
parser = Parser(plant, scene_graph)
parser.AddModelsFromUrl("package://drake_models/iiwa_description/urdf/iiwa14_primitive_c
plant.Finalize()
diagram = builder.Build()
```

https://drake.mit.edu/doxygen_cxx/classdrake_1_1multibody_1_1_multibody_plant.html

https://drake.mit.edu/doxygen_cxx/classdrake_1_1geometry_1_1_scene_graph.html

Link, Joints, Frames



Analytical Expressions of Serial Manipulator Jacobians and their High-Order Derivatives based on Lie Theory, Fu et al, ICRA 2020

URDF - Robot Description Files

```
<?xml version="1.0"?>
<sdf version="1.7">
  <model name="iiwa14">
    <link name="iiwa_link_0">
      <inertial>
        <pose>-0.1 0 0.07 0 0 0</pose>
        <mass>5</mass>
        <inertia>
          <ixx>0.05</ixx>
          <ixy>0</ixy>
          <ixz>0</ixz>
          <iyy>0.06</iyy>
          <iyz>0</iyz>
          <izz>0.03</izz>
        </inertia>
      </inertial>
      <visual name="iiwa_link_0_fixed_joint_lump__iiwa_link_0_visual">
        <geometry>
          <mesh>
            <scale>1 1 1</scale>
            <uri>../meshes/visual/link_0.obj</uri>
          </mesh>
        </geometry>
```

SDF (Simulation Description Format)

```
<?xml version="1.0" ?>
<sdf version="1.6">
  <model name="block">
    <!-- Set to true if you want it fixed in space; omit or set false for a free body
    <!-- <static>true</static> -->

    <link name="block_link">
      <!-- Inertial properties (centered at link frame) -->
      <inertial>
        <!-- Pose of the inertia frame relative to link frame -->
        <pose>0 0 0 0 0 0</pose>
        <!-- Mass in kg -->
        <mass>1.0</mass>
        <!-- Inertia tensor (kg·m^2) for a 0.2 x 0.1 x 0.05 m box -->
        <!-- Ixx = 1/12 m (y^2 + z^2) = 0.0010416667 -->
        <!-- Iyy = 1/12 m (x^2 + z^2) = 0.0035416667 -->
        <!-- Izz = 1/12 m (x^2 + y^2) = 0.0041666667 -->
        <inertia>
          <ixx>0.0010416667</ixx>
          <ixy>0.0</ixy>
          <ixz>0.0</ixz>
          <iyy>0.0035416667</iyy>
          <iyz>0.0</iyz>
          <izz>0.0041666667</izz>
        </inertia>
      </inertial>
```

SDF - with meshes

```
<?xml version="1.0" ?>
<sdf version="1.6">
  <model name="block">
    <!-- Set to true if you want it fixed in space; omit or set false for a free body
    <!-- <static>true</static> -->

    <link name="block_link">
      <!-- Inertial properties (centered at link frame) -->
      <inertial>
        <!-- Pose of the inertia frame relative to link frame -->
        <pose>0 0 0 0 0 0</pose>
        <!-- Mass in kg -->
        <mass>1.0</mass>
        <!-- Inertia tensor (kg·m^2) for a 0.2 x 0.1 x 0.05 m box -->
        <!-- Ixx = 1/12 m (y^2 + z^2) = 0.0010416667 -->
        <!-- Iyy = 1/12 m (x^2 + z^2) = 0.0035416667 -->
        <!-- Izz = 1/12 m (x^2 + y^2) = 0.0041666667 -->
        <inertia>
          <ixx>0.0010416667</ixx>
          <ixy>0.0</ixy>
          <ixz>0.0</ixz>
          <iyy>0.0035416667</iyy>
          <iyz>0.0</iyz>
          <izz>0.0041666667</izz>
        </inertia>
      </inertial>
```

Meshes: block.obj

```
# Block 0.2 x 0.1 x 0.05 (m), centered at origin
mtllib block.mtl
usemtl Blue

# Half-extends: hx=0.1, hy=0.05, hz=0.025
v -0.1 -0.05 -0.025
v  0.1 -0.05 -0.025
v  0.1  0.05 -0.025
v -0.1  0.05 -0.025
v -0.1 -0.05  0.025
v  0.1 -0.05  0.025
v  0.1  0.05  0.025
v -0.1  0.05  0.025

# Faces (triangles), counter-clockwise with outward normals

# Bottom (z = -0.025)
f 1 2 3
f 1 3 4

# Top (z = +0.025)
f 5 6 7
f 5 7 8

# +Y face (front, y = +0.05)
f 4 3 7
```

https://drake.mit.edu/doxygen_cxx/group__multibody__parsing.html

https://manipulation.mit.edu/robot.html#example2

Model Directives

- Load models (add_model)
  - package://drake_models holds many useful models
- Load other directives (add_directives)
- Load predefined controllers (model_drivers)
- Specify plant and simulation settings (e.g. timestep) (plant_config)

Model Directives

```
scenario = """
directives:
- add_directives:
    file: package://manipulation/clutter.dmd.yaml
- add_model:
    name: foam_brick
    file: package://drake/examples/manipulation_station/models/061_foam_brick.sdf
    default_free_body_pose:
        base_link:
            translation: [0, -0.6, 0.2]
"""

builder = DiagramBuilder()
scenario = LoadScenario(data=scenario_string)
station = builder.AddSystem(MakeHardwareStation(scenario, meshcat=meshcat))
...
diagram = builder.Build()
```

https://manipulation.mit.edu/robot.html#example8

https://drake.mit.edu/doxygen_cxx/classdrake_1_1systems_1_1controllers_1_1_inverse_dynamics_controller.html

https://manipulation.mit.edu/robot.html#example6